A Method of Fault Simulation Based on Stem Regions

FADI MAAMARI AND JANUSZ RAJSKI, MEMBER, IEEE

Abstract—**An exact fault simulation can be achieved by simulating only the faults on reconvergent fan-out stems, while determining the detectability of faults on other lines by critical path tracing within fan-out-free regions. We have delimited, for every reconvergent fan-out stem, a region of the circuit outside of which the stem fault does not have to be simulated. Lines on the boundary of such a stem region, called exit lines, have the following property: if the stem fault is detected on the line, and the line is critical with respect to a primary output, then the stem fault is detected at that primary output. Any fault simulation technique can be used to simulate the stem fault within its stem region. The fault simulation complexity of a circuit is shown to be directly related to the number and size of stem regions in the circuit.**

We have used the concept of stem regions as a framework for an efficient fault simulator for combinational circuits. The concept allows a static reduction of the circuit area of explicit analysis for single as well as multiple output circuits. A dynamic reduction of processing steps is also achieved as the fault simulation progresses and fault coverage increases. Both the static and dynamic reductions are fully compatible with the parallel pattern evaluation technique, resulting in a very efficient implementation. The simulation algorithm is described, and experimental results are shown for the well-known benchmark circuits.

I. Introduction

THE GROWING use of scan path designs in VLSI circuits has increased the interest in fault simulation methods for combinational circuits [1], [2], [4]. The restriction to combinational circuits has the advantage that in such circuits, only faults on reconvergent fan-out stems have to be explicitly simulated, while the detectability of the other faults is determined by simulating them locally within a fan-out-free region [9], or, more efficiently, by critical path tracing [1], [2], [4]. Recently, a single fault propagation method using the concept of dominators was proposed [6]. The method, which propagates faults to their immediate dominator instead of the primary outputs, is efficient for single output circuits, but does not fully exploit the topology of multiple output circuits [8].

In this paper, we propose a test vector independent reconvergent fan-out analysis, the implications of which are twofold: it can be applied to any fault simulation technique to reduce fault simulation time by processing stem faults only within limited regions of the circuit, and it provides a measure of the fault simulation complexity of a circuit by computing the average stem region overlap on each circuit line.

In the proposed analysis, we determine, for every reconvergent fan-out stem, a region of the circuit called the *stem region* [10]. The stem region is bounded by lines that have the following property: when the stem fault is detectable on the line, and the line is critical, i.e., a change in its logic value changes the logic value of a primary output, then the stem fault is detectable on that primary output. As a consequence of this property, stem faults need not be explicitly simulated beyond these special lines, called *exit lines*, as opposed to being propagated all the way to primary outputs, as is usually done in exact fault simulation techniques.

Each circuit line can be part of several stem regions that overlap each other. The degree of region overlap corresponds to the maximum number of stem faults that may have to be processed on that line and, when computed for all the lines in the circuit, leads to an accurate estimate of the expected worst-case fault simulation time for that circuit. This estimate is a good measure of the fault simulation complexity of the circuit, as it is test vector independent.

The paper also describes the use of the stem region concept as a framework for Tulip, a fast fault simulator for combination circuits. Tulip extracts the stem regions in a preprocessing phase, integrating them into the circuit description structure to achieve a static reduction of processing steps. In addition, as the fault simulation progresses and fault coverage increases, dropping schemes enable a dynamic reduction of processing steps. Both the static and dynamic reductions are fully compatible with the parallel pattern evaluation technique, resulting in a very efficient implementation.

II. The Stem Regions

We will define the regions in the context of a single or multiple output combinational circuit that can be described as a set of nodes connected by a set of lines, where a node can be a fan-out stem, a gate, a primary input, or a primary output. A point a in the circuit will be considered as reaching a point b in the circuit if a logic signal can travel from point a to point b. Note that a point reaches itself.

Manuscript received February 21, 1989; revised June 20, 1989. This work was supported by grants from the Natural Science and Engineering Research Council of Canada and the Quebec Fonds pour la Formation de Chercheurs et l'Aide à la Recherche. This paper was recommended by Editor M. R. Lightner.

The authors are with the VLSI Design Laboratory, Department of Electrical Engineering, McGill University, Montreal, P.Q., Canada H3A 2A7.

IEEE Log Number 8931874.

0278-0070/90/0200-0212$01.00 © 1990 IEEE

We begin by identifying two types of reconvergence gates for reconvergent fan-out stems: primary reconvergence gates (Definition 1) and secondary reconvergence gates (Definition 2). Primary and secondary reconvergence gates are used to delimit the stem regions (Definition 3) and their exit lines (Definition 4). Lemma 1 and Corollary 1 demonstrate that the circuit subnetworks driven by the exit lines of a stem region are disjoint. This leads to Theorem 1, which describes the relation between the detectability of a reconvergent fan-out stem fault at primary outputs, the detectability of the stem fault at its exit lines, and the criticality of the exit lines.

Definition 1: If there are two or more disjoint paths between a stem A and a gate B, then A is a reconvergent fan-out stem, and B is a *primary reconvergence gate* for stem A. Otherwise, stem A is a non-reconvergent fan-out stem.

Two circuits are used throughout this section to illustrate the various definitions that are presented. In the circuit of Fig. 1, $N1$ and $N6$ are reconvergent fan-out stems, while $N5$ and $N10$ are non-reconvergent fan-out stems. Gates $N7$ and $N11$ are primary reconvergence gates for stem $N1$, and gate $N12$ is a primary reconvergence gate for stem $N6$. In the other circuit, shown in Fig. 2, $N1$ and $N5$ are both reconvergent fan-out stems, and $N4$ and $N8$ are primary reconvergence gates for stems $N1$ and $N5$, respectively. Notice that $N8$ is not a primary reconvergence gate for stem $N1$, as there are no two disjoint paths between $N1$ and $N8$.

If there are no reconvergent fan-out stems on the paths between a reconvergent fan-out stem A and its primary reconvergence gates, then A is a *narrow reconvergent fan-out stem*. Otherwise, A is a *wide reconvergent fan-out stem*.

Stem $N6$ is the only narrow reconvergent fan-out stem in Fig. 1. $N6$ is on a path between stem $N1$ and $N11$, a reconvergence gate of $N1$, therefore, $N1$ is a wide reconvergent fan-out stem. In Fig. 2, both $N1$ and $N5$ are narrow reconvergent fan-out stems.

We will now use a narrow reconvergent fan-out stem as the starting point of an inductive definition.

Definition 2: Let A be a narrow reconvergent fan-out stem.

a) If A is located on a path between a reconvergent fan-out stem B and a primary reconvergence gate of B, then all the primary reconvergence gates of stem A that are not primary reconvergence gates of stem B are *secondary reconvergence gates* for stem B.
b) If stem B is located on a path between a reconvergent fan-out stem C and a primary reconvergence gate of C, then all the primary and secondary reconvergence gates of stem B that are not primary reconvergence gates of stem C are *secondary reconvergence gates* for stem C.

Reconvergent fan-out stem $N6$ of Fig. 1 is located on a path between stem $N1$ and $N11$, a primary reconvergence gate of $N1$. Therefore, all primary reconvergence gates of stem $N6$ ($N12$ is the only one) and all secondary reconvergence gates of stem $N6$ (there are none) are secondary reconvergence gates for stem $N1$.

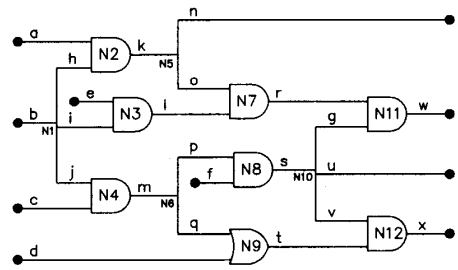

Fig. 1. A combinational circuit with reconvergent fan-out.

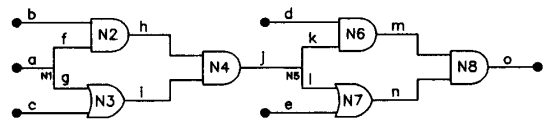

Fig. 2. A circuit with two disjoint stem regions: f, g, h, i, j, and k, l, m, n, o.

A primary or secondary reconvergence gate of a stem A that does not drive another primary or secondary reconvergence gate of A is a *closing reconvergence gate* for A. $N11$ and $N12$ are closing reconvergence gates for stem $N1$ in Fig. 1. $N12$ also is a closing reconvergence gate for stem $N6$. In the circuit of Fig. 2, $N4$ and $N8$ are closing reconvergence gates for stems $N1$ and $N5$, respectively. Note that as a circuit can have multiple outputs, a stem can have more than one closing reconvergence gate.

Definition 3: The *stem region* of a reconvergent fan-out stem A is composed of all the circuit nodes that satisfy conditions a) and b),

a) they are reached by the reconvergent fan-out stem node A;
b) they reach a closing reconvergence gate of stem A;

and of all the output lines of these nodes (fan-out branches of a stem and output of a gate).

There are two stem regions in Fig. 1. The stem region of $N1$ is composed of lines h, i, j, k, l, m, n, o, p, q, r, s, t, g, u, v, w, and x, and of nodes $N1$–$N12$. The stem region of $N6$ is composed of lines p, q, s, t, g, u, v, and x, and of nodes $N6$, $N8$, $N9$, $N10$, and $N12$. Two stem regions are also present in Fig. 2, where they are disjoint, as opposed to the stem regions of Fig. 1 that overlap each other. The stem region of $N1$ is composed of lines f, g, h, i, and j, and of nodes $N1$–$N4$. The stem region of $N5$ is composed of lines k, l, m, n, and o, and of nodes $N5$–$N8$.

Definition 4: Let x be a line in the stem region of a stem A. x is an exit line for stem A if the following two conditions are met:

a) x is an output line of a node that is in the stem region of A.

b) x is an input line of a node that is not in the stem region of A.

The exit lines of a stem region are the lines through which a logic signal can exit the stem region. In Fig. 1, n, w, u, and x are exit lines for stem $N1$. g, u, and x are exit lines for stem $N6$. The two reconvergent fanout stems in Fig. 2 have one exit line each: j is an exit line for stem $N1$ and o is an exit line for stem $N5$.

Lemma 1: If x and y, $x \neq y$, are exit lines of a reconvergent fan-out stem A, then there is no line in the circuit that is reached by both x and y.

Proof (by contradiction): If x and y both reach a line z, then x and y must reconverge at a reconvergence gate C preceding z (Fig. 3). This implies that C, as a primary or secondary reconvergence gate of A, is in the stem region of A, which is a contradiction, since x and y, as exit lines of A, cannot reach an element of its stem region. A detailed proof is given in [11]. ■

Corollary 1: If x and y, $x \neq y$, are exit lines of a reconvergent fan-out stem A, C_x, the circuit subnetwork driven by x, and C_y, and the circuit subnetwork driven by y, are disjoint.

The effect of Corollary 1 on the exit lines of a stem A is shown in Fig. 4.

Theorem 1: Let x be an exit line of a stem A through which A reaches primary output o. For a given input pattern t applied to the circuit, a single fault f_A on stem A is detected on o if and only if:

a) f_A is detected on x, and
b) x is critical with respect to o (a single fault f_x on line x that causes a faulty logic value on x causes a faulty logic value on o).

Proof: Theorem 1 is a direct consequence of Corollary 1. A detailed proof is given in [11].

Using Theorem 1, we can define a fault simulation framework in which there is no need to explicitly simulate stem faults beyond their respective stem regions. This result is general, as any fault simulation technique can be used to explicitly simulate the stem faults within their respective stem regions.

A. Extraction of the Stem Regions

A four-step algorithm, described in Appendix A, is used to extract the stem regions, as follows. For every stem s_i in the circuit, in a reversed levelized order:

Step 1 (s_i): The lines driven by s_i are tagged in a forward propagation through the levelized circuit, identifying the primary reconvergence gates of s_i. If no primary reconvergence gates are found, s_i is non-reconvergent.

Step 2 (s_i): Backtracing from the reached circuit outputs, closing reconvergence gates are identified. Each reconvergent fan-out stem s_j located between s_i and a primary reconvergence gate of s_i is checked for secondary reconvergence.

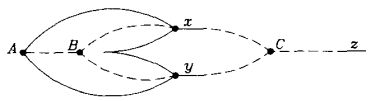

Fig. 3. Hypothesis of Lemma 1: a line z is reached by both x and y.

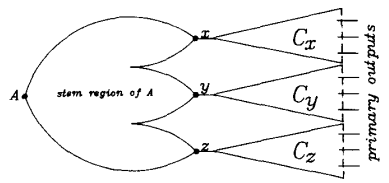

Fig. 4. Disjoint subnetworks driven by exit lines.

Step 3 (s_i): If secondary reconvergence is found, backtracing from the reached circuit outputs is performed again, identifying and storing the closing reconvergence gates of s_i, taking into account secondary reconvergence.

Step 4 (s_i): Exit lines of s_i are gathered and stored.

The CPU time required to extract the stem regions of the benchmark circuits is given in Table I.

III. The Fault Simulation Framework

The fault simulation problem can be defined as the determination, for each fault f in a fault set F, of the detectability of f when an input pattern t is applied to the circuit. Let $d(f)$ be a Boolean variable representing the detectability of f, with value 1 if f is detected and 0 if it is not. A fault f is detected when its fault effect propagates to at least one primary output:

$$d(f) = d^{p_1}(f) + d^{p_2}(f) + \cdots + d^{p_n}(f) \quad (1)$$

where $p_1, p_2, \cdots, p_n$ are the primary outputs of the circuit and $d^{p_i}(f)$ represents the detectability of fault f on primary output p_i. Let $\{l_1, \cdots, l_m\}$ be the set of lines in the circuit, where a line is either a primary output, a gate input or a fan-out stem. Assuming the single stuck-at fault model, where l_{i0} and l_{i1} are the stuck-at 0 and 1 faults on l_i:

$$F = \{l_{i0}, l_{i1}, i = 1, \cdots, m\}. \quad (2)$$

Let L represent the fault-free line value on line l. For each line l, $d(l_L) = 0$, and the fault simulation problem is reduced to that of determining $d(l_{\bar{L}})$, which we will represent as $d(l_*)$ from here on, for each line l. $d(l_*)$ will be determined differently, depending on whether l is (a) a primary output, (b) a gate input, (c) a non-reconvergent fan-out stem, or (d) a reconvergent fan-out stem.

a) If l is a primary output, $d(l_*)$ is simply equal to 1.
b) If l is a gate input, $d(l_*)$ is determined by critical path tracing [1], formalized as follows: if o is the gate output and $d^o(l_*)$ is the Boolean variable representing the detectability on o of l_*, then

$$d(l_*) = d^o(l_*) \cdot d(o_*). \quad (3)$$

TABLE I
PERFORMANCE FIGURES ON A SUN WORKSTATION

	CPU time (sec) Sun3/260		
circuits	Region Extraction	Fault Simulation (100K patterns)	f.c.
C432	0.6	31.0	0.9884
C499	0.5	44.2	0.9920
C880	0.7	16.3*	1.0000
C1355	1.6	124.9	0.9970
C1908	3.6	138.5	0.9971
C2670	1.6	338.5	0.8620
C3540	7.7	810.3	0.9641
C5315	3.2	473.4	0.9942
C6288	49.8	2182.6	0.9946
C7552	8.0	1076.2	0.9799

* (for 24320 patterns)

If the $d()$'s are computed in a reversed levelized order, then $d(o_*)$ is already known, and $d(l_*)$ can be determined in one step (the propagation of a fault from a line to its successor will be considered as a single processing step).

c) If l is a non-reconvergent fan-out stem, its fan-out branches are independent [2] (they do not reconverge) and

$$d(l_*) = d^{f_1}(l_*) + d^{f_2}(l_*) + \cdots + d^{f_n}(l_*) \quad (4)$$

where $f_1, f_2, \cdots, f_n$ are the fan-out branches of l.

d) If l is a reconvergent fan-out stem, $d(l_*)$ is determined using the stem region concept, through Theorem 2.

Theorem 2: Let $\{x_1, x_2, \cdots, x_n\}$ be the set of exit lines of fan-out stem l.

$$d(l_*) = d^{x_1}(l_*) \cdot d(x_{1*}) + d^{x_2}(l_*) \cdot d(x_{2*}) + \cdots + d^{x_n}(l_*) \cdot d(x_{n*}). \quad (5)$$

Proof: Let C_x be the circuit subnetwork driven by line x.

Let $p_0, p_1, \cdots, p_N$ be the N primary outputs reached by stem l, where $p_0, \cdots, p_{m_1}$ are in C_{x_1}, $p_{m_1+1}, \cdots, p_{m_2}$ are in C_{x_2}, $\cdots$, and $p_{m_{n-1}+1}, \cdots, p_N$ are in C_{x_n} (Fig. 5).

$$\begin{aligned} d(l_*) &= d^{p_1}(l_*) + d^{p_2}(l_*) + \cdots + d^{p_{m_n}}(l_*) \\ &\qquad \text{(from (1))} \\ &= d^{x_1}(l_*) \cdot d^{p_1}(x_{1*}) + \cdots \\ &\quad + d^{x_1}(l_*) \cdot d^{p_{m_1}}(x_{1*}) + d^{x_2}(l_*) \cdot d^{p_{m_1+1}}(x_{2*}) \\ &\quad + \cdots + d^{x_2}(l_*) \cdot d^{p_{m_2}}(x_{2*}) \\ &\quad + \cdots + d^{x_n}(l_*) \cdot d^{p_{m_{n-1}+1}}(x_{n*}) + \cdots \\ &\quad + d^{x_n}(l_*) \cdot d^{p_N}(x_{n*}) \qquad \text{(using Theorem 1)} \end{aligned}$$

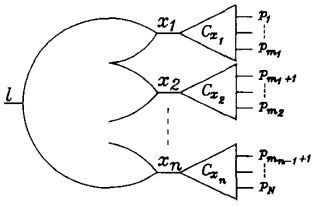

Fig. 5. Primary outputs reached by a stem through its exit lines.

$$\begin{aligned} &= d^{x_1}(l_*) \cdot \left(d^{p_1}(x_{1*}) + \cdots + d^{p_{m_1}}(x_{1*})\right) \\ &\quad + d^{x_2}(l_*) \cdot \left(d^{p_{m_1+1}}(x_{2*}) + \cdots + d^{p_{m_2}}(x_{2*})\right) \\ &\quad + \cdots + d^{x_n}(l_*) \cdot \left(d^{p_{m_{n-1}+1}}(x_{n*}) + \cdots \right. \\ &\quad \left. + d^{p_N}(x_{n*})\right) \\ &= d^{x_1}(l_*) \cdot d(x_{1*}) + d^{x_2}(l_*) \cdot d(x_{2*}) + \cdots \\ &\quad + d^{x_n}(l_*) \cdot d(x_{n*}) \end{aligned}$$

■

Again, if the $d()$'s are computed in a reversed levelized order, then the $d(x_{i*})$'s are already known. Computing $d^{x_i}(l_*)$, for $i = 1, \cdots, n$ involves the propagation of the stem fault within the stem region to its exit lines, a task that will require at most S processing steps, where S is the number of lines in the stem region:

$$S = \text{Size}(C_l) - \text{Size}(C_{x_1} \cup C_{x_2} \cup \cdots \cup C_{x_n}). \quad (6)$$

The second component of (6) represents the static reduction of the circuit area in which the stem fault is explicitly simulated. Although it varies with circuit topology, experimental results show the reduction to be significant in most benchmark circuits (Section VI).

A. Fault Simulation Complexity

Stem regions may overlap each other. In Fig. 1, the stem regions of $N1$ and $N6$ overlap each other on lines p, q, s, t, g, u, v, and x. The region overlap on these lines is therefore 2. Lines h, i, j, k, l, m, n, o, r, and w have an overlap value of 1, and lines a, b, c, d, e, and f have an overlap of 0.

The worst-case fault simulation time of a circuit is related to the number of circuit lines and the average stem region overlap on the circuit lines. Let t be the time required to process a fault on a line i during fault simulation, and R_i the region overlap on that line. There can be no more than R_i faults to process on that line. Assuming it will also take t time to determine the criticality of the line, it will take at most $t(R_i + 1)$ time to process the line. We can now express T, the maximum time it will take to fault simulate the whole circuit:

$$T = \sum_{i=1}^{N} t(R_i + 1) \quad (7)$$

where N is the number of lines in the circuit. It can also be expressed as

$$T = Nt \sum_{i=1}^{N} \frac{R_i}{N} + \sum_{i=1}^{N} t = Nt(\bar{R} + 1) \tag{8}$$

where $\bar{R}$ is the average region overlap in the circuit. Therefore, the average region overlap plus one, multiplied by the number of circuit lines, measures the fault simulation complexity of the circuit. $\bar{R}$ reflects the nonlinear growth of the fault simulation complexity. All reconvergent fan-out free circuits, for example, have an average overlap $\bar{R}$ equal to zero, and can be fault simulated in linear time.

B. Parallel Pattern Evaluation

Both the fault-free simulation and the fault simulation within the proposed framework involve operations on Boolean variables, and can be done for many input vectors simultaneously (typically the wordlength of the machine, which happens to be 32 in our case), significantly increasing the fault simulation efficiency. The simulation will then consist of determining, for a vector V of input patterns, the vector of detectabilities $D(l_*)$ of each line l in the circuit, where

a) if l is a primary output, then

$$D(l_*) = (11 \cdots 1) \tag{9}$$

b) if l is a gate input, then

$$D(l_*) = D^o(l_*) \cdot D(o_*) \tag{10}$$

where o is the gate output;

c) if l is a non-reconvergent fan-out stem;

$$D(l_*) = D^{f_1}(l_*) + D^{f_2}(l_*) + \cdots + D^{f_n}(l_*) \tag{11}$$

where $f_1, f_2, \cdots, f_n$ are the fan-out branches of l;

d) if l is a fan-out stem, then

$$D(l_*) = D^{x_1}(l_*) \cdot D(x_{1*}) + D^{x_2}(l_*) \cdot D(x_{2*}) + \cdots + D^{x_n}(l_*) \cdot D(x_{n*}) \tag{12}$$

where $x_1, x_2, \cdots, x_n$ are the exit lines of x.

IV. The Fault Simulation Algorithm

The fault simulation algorithm has two main components. The stem regions are extracted in a preprocessing step, using the region extraction algorithm described in Section VI. Then, for each vector V of input patterns, the simulator performs one forward pass and one backward pass through the circuit. In the forward pass, fault-free line values are computed, and the stem faults are propagated to their exit lines:

forward_pass()
perform fault-free simulation
for each fanout stem l with exit lines $x_1, x_2, \cdots, x_n$:
  compute $D^{x_i}(l_*)$, $i = 1, \cdots, n$
  store $D^{x_i}(l_*)$ on line x_i.
endfor

In the backward pass, the $d()$'s are computed in a reversed levelized order, completing the simulation for the vector V of input patterns.

backward_pass()
for each circuit line l_i in a decreased levelized order, starting at the primary outputs:
  if l_i is a primary output:
    $D(l_{i*}) = (11 \cdots 1)$
  elseif l_i is a gate input:
    $D(l_{i*}) = D(o_*) \cdot D^o(l_{i*})$
  elseif l_i is a non-reconvergent fan-out stem:
    $D(l_*) = D^{f_1}(l_*) + D^{f_2}(l_*) + \cdots + D^{f_n}(l_*)$
  endif
  for each stem l_j for which l_i is an exit line,
    $D(l_{j*}) = D(l_{j*}) + D^{l_i}(l_{j*}) \cdot D(l_{i*})$
  endfor
endfor

Notice that if l is a fan-out stem, $D(l_*)$ is computed when the exit lines of l are processed. Figs. 6 and 7 follow, for one input pattern, the processing of a fan-out stem fault l_* through the forward and backward passes. In Fig. 6, l_* is propagated within the stem region during the forward pass, reaching exit lines x_1 and x_3, but not x_2. Thus l is therefore, stored on x_1 and x_3, and a 0 on x_2. Fig. 7 shows the backward pass, where $d(l_*)$ is marked as "1" when exit line x_1 is processed.

V. Dynamic Reduction of Processing Steps

A. Fault Dropping

In many fault simulation applications, such as test pattern generation and test set fault coverage computation, there is no need to determine the detectability of a fault if the fault has been detected by a previously applied vector. Let $L = \{l_1, l_2, \cdots, l_n\}$ be the set of circuit lines. Let $T = \{t_1, t_2, \cdots, t_n\}$ be an ordered set of input patterns. The fault simulation problem can then be expressed as follows.

For each $l \in L$, determine $d(l_*)$ for input pattern t_j if $d(l_*) = 0$ for input patterns $t_1, t_2, \cdots, t_{j-1}$.

In the more general fault simulators [3], [16], [13], [14], where each circuit fault is propagated to the primary outputs, a fault f can be dropped from further processing as soon as it is detected. In simulators where the detectability of a fault is determined transitively from the detectability of a fault of a higher level (surrogate faults [9], stem regions), an additional condition appears for a fault f to be dropped: all the faults the detectability of which is determined based on the detectability of f must also have been detected and dropped. Within the fault simulation framework based on stem regions, there are three types of transitive dependencies between fault detectabilities,

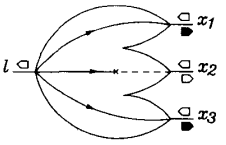

Fig. 6. Forward pass.

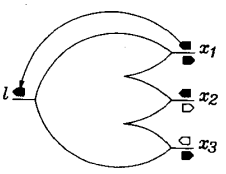

Fig. 7. Backward pass.

expressed in (3)–(5). They can be described by edges in a directed graph $G = (V, E)$, where the vertices of G represent the circuit lines. An initial graph containing all the transitive dependencies can be constructed through the following procedure:

```
construct_dependency_graph()
for each line (vertex) l in the circuit (graph):
   if l is a primary output:
      no dependency edge
   elseif l is a gate input:
      add edge from l to gate output
   elseif l is a non-reconvergent fanout stem
      for each fanout branch f_i of l,
         add edge from l to f_i
      endfor
   elseif l is a reconvergent fanout stem
      for each exit line x_i of l,
         add edge from l to x_i
      endfor
   endif
endfor
```

The initial dependency graph G of the circuit of Fig. 2 is shown in Fig. 8.

The fault dropping, viewed as the pruning of the set L of lines l for which $d(l_*)$ has to be determined, can then be performed by pruning the initial graph G, as follows.

Conditions for Dropping Line l from Set L

a) both stuck-at-0 and stuck-at-1 faults on l are detected, and

b) there are no incoming edges on vertex l in the dependency graph G of the circuit.

Dropping Line l from Set L

a) remove line l from set L, and

b) remove all edges coming out of vertex l in the dependency graph G of the circuit.

B. Reduction of the Fault-Free Simulation

Fault-free simulation is necessary to determine whether a fault effect reaches a primary output. As the fault simulation progresses, however, and the fault dropping spreads, it may become unnecessary to perform fault-free simulation in parts of the circuit. Let C_l be the circuit subnetwork driven by line l. When all the faults in the subnetwork driving the primary outputs reached by l have been detected and dropped (Fig. 9), C_l can be dropped from the fault-free simulation in the forward pass.

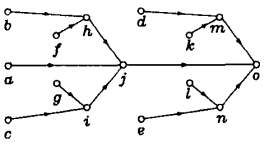

Fig. 8. Initial dependency graph of the circuit of Fig. 2.

Let F be the set of all the circuit lines that can be dropped from the fault-free simulation. We can keep track of F by calling the following procedure, each time a line l is dropped from the set L of lines for which $d(l_*)$ has to be determined:

```
check_ffs_dropping(l)
if ((l is a primary output line) or
   (all the lines fed by l are in F))
   add l to F;
   for(each line feeding l)
      check_ffs_dropping(line);
   endfor
endif
```

The computational overhead spent to keep track of F is negligible, as activity (growth of F) is triggered only when a primary output line is dropped from the set L of lines for which $d(l_*)$ has to be determined. The amount of processing saved through fault-free simulation dropping depends on several factors:

- The size and topology of the circuit. Fault-free simulation dropping is particularly relevant in the fault simulation of large scan designs, where circuits are often very wide and not very deep. In such circuits, it is not unusual to see the fault simulation progress to a point where only a few isolated pockets of faults remain undetected. At that point, fault-free simulation represents a significant portion of the total processing time, and should not be done unnecessarily.
- The presence of random-pattern-resistant faults. In easily random-testable circuits, the interval between the spread of the fault-free simulation dropping and the coverage of all faults is sometimes too small for the effect of the dropping to be noticeable.
- The presence of redundant faults. Redundant faults limit the spread of the fault-free simulation dropping, as they are undetectable.

Except for C880, which is easily random-testable, all the benchmark circuits contain redundant faults. In C880, half the circuit is dropped from fault-free simulation by the end of the fault simulation, reducing total processing time by 15 percent. An interesting circuit is C2670, where

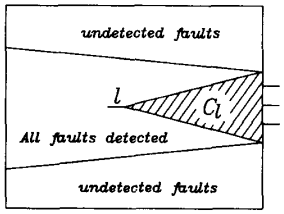

Fig. 9. Reduction of the fault-free simulation.

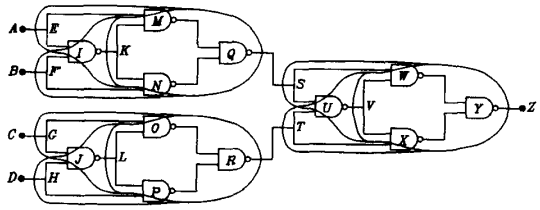

Fig. 10. Three expanded EX-OR gates connected in a tree-like fashion.

3.2 million random patterns are required to get full coverage of the detectable faults. In that circuit, the presence of the redundant faults limits the fault-free simulation dropping to 5 percent of the circuit. Without them, however, two thirds of the circuit is dropped from fault-free simulation after only 4000 patterns, reducing total processing time for 3.2 million patterns by 43 percent.

VI. Experimental Results

To enable us to measure the improvement in fault simulation efficiency resulting from the stem region analysis, the average overlap ($\bar{R}$) values and maximum fault simulation time values (T) were computed for the case when stem faults are explicitly simulated all the way to primary outputs, as well as for our proposed method, where stem faults are explicitly simulated only within their respective stem regions.

The results, presented in Table II, show that the reduction in average overlap that is achieved with the stem region analysis varies with circuit topology.

In C1355, an order of magnitude reduction is observed, as most fan-out stems in that circuit are part of expanded Exclusive-OR gates and reconverge very quickly. The reduction is apparent in the circuit structure of Fig. 10. The figure represents a three Exclusive-OR gate tree and its stem regions. Each Exclusive-OR gate is implemented in an expanded form, using four NAND gates and three reconvergent fan-out stems. Using the stem region analysis, the three fan-out stem faults of an Exclusive-OR gate, whatever the position of the gate in the circuit, will be simulated only to the output of the Exclusive-OR gate, instead of being simulated all the way to the primary outputs of the circuit.

A much smaller improvement is observed in C6288, where most fan-out stems reconverge at primary outputs. In many other circuits, the average overlap is reduced by half.

The results also show the striking difference in fault simulation complexity that can exist in circuits of similar size: C6288 with an $\bar{R}$ of 184.9 and C7552 with an $\bar{R}$ of 10.9. A distribution of the circuit lines according to their overlap value is shown for these circuits in the histograms of Figs. 11, 12.

We can also see in Table II that circuits with a large number of inputs and outputs—a characteristic of scannable circuits—such as C2670, C5315, and C7552 have an average overlap similar to the one observed in smaller circuits C499, C880, and C1355. As a result, differences in the T value of these circuits are mainly due to the differences in their number of lines. At the opposite line of the spectrum, the region overlap for C6288 is a lot higher than any of the other circuits.

An interesting comparison can be made between the re-

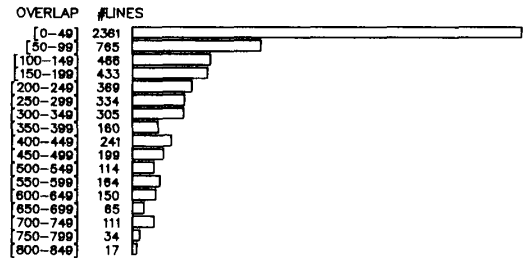

Fig. 11. Histogram of region overlap for circuit 6288.

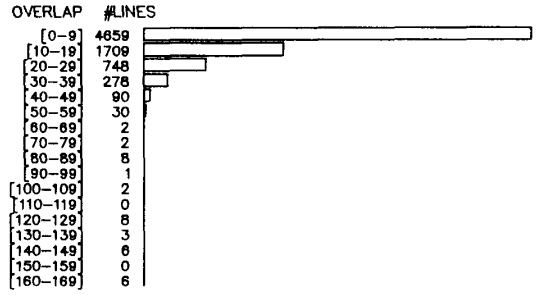

Fig. 12. Histogram of region overlap for circuit 7552.

TABLE II
Summary of Experimental Results

Benchmark Circuit Characteristics								
A: stems simulated within stem regions								
B: stems simulated to primary outputs								
					$\bar{R}$		T (in t units)	
circuit	stems	gates	PIs	POs	A	B	A	B
C432	89	160	36	7	29.5	30.3	13165	13537
C499	59	202	41	32	11.5	17.7	6221	9325
C880	125	383	60	26	8.3	16.1	8207	15028
C1355	259	546	41	32	7.4	60.5	11389	83381
C1908	385	880	33	25	19.7	54.7	39542	106335
C2670	454	1193	233	140	10.3	14.8	30213	42179
C3540	579	1669	50	22	44.2	55.1	159990	200047
C5315	806	2307	178	123	7.2	15.7	43565	88746
C6288	1456	2416	32	32	184.9	203.0	1168668	1282833
C7552	1300	3512	207	108	10.9	25.7	89983	201478

sults of circuits C499 and C1355, which are functionally equivalent, the difference being that the Exclusive-OR gates in C499 have been expanded in C1355. Although C1355 has close to three times the number of lines in C499, its average overlap is smaller, and it will take only twice as much time to fault simulate it as it will take for C499.

Tulip is implemented in C on a Sun workstation. Experimental results, obtained for the benchmark circuits [5] using fault dropping, confirm the high performance of the simulator (Table I). Circuit C2670 (2670 lines), for example, is preprocessed and fault simulated for 102 400 vectors in less than 6 min of CPU.

VII. Conclusion

The concept of stem regions is general, as it is test vector independent and can be used with any fault simulation technique to speedup simulation. For both the region extraction algorithm and the fault simulation using the stem region concept, we observe an $O(n^2)$ complexity on the matrix multiplication decoder [7] that might become a benchmark circuit for worst-case analysis of fault simulation algorithms. This is due to the fact that the average overlap in the decoder is of $O(n)$ order. Although this may seem discouraging, there is no known exact fault simulation algorithm that achieves better complexity on that circuit, and most practical circuits are far from showing such worst-case behavior. In fact, in the results we obtain for the benchmark circuits, only the 16-bit multiplier C6288 stands out as having an exceedingly high average overlap, while the other circuits, including some large scan designs, typically have a low number of faults to process on each of their lines.

While we have only shown how the stem regions and exit lines can be used advantageously in the context of fault simulation, it would be interesting to explore whether their topological properties could be exploited in other applications related to the propagation of logic signals along potentially reconvergent paths.

Appendix A
Region Extraction Algorithm

```
Step1(STEM)
beginstep
for(each fanout branch of STEM)
   if(the fanout branch is a primary output)
      push the stem in output_stack;
   elseif
      assign a new tag to the fanout branch;
      line_status of fanout branch = STEM;
      put the node it drives in nodes_to_process list;
   endif
endfor
while(there is more than one node in nodes_to_process list)
   fetch lowest level node from nodes_to_process list;
   if(node is a stem)
      fob_counter of stem = number of fan. br. of the stem;
      node_status of stem = ENCOUNTERED;
      for(every fanout branch of the stem)
         assign the stem tag to the fanout branch;
         if(the fanout branch is a primary output)
            push the stem in output_stack;
         else
            line_status of fanout branch = STEM;
            put the node driven by the fanout branch in
            nodes_to_process list;
         endif
      endfor
      remove stem tag;
   elseif(node is a gate)
      node_status of gate = ENCOUNTERED;
      if(at least two gate inputs have different tags)
         node_status of the gate = PRIM_RECONV_GATE;
         set prg_flag;
         assign a new tag to gate output;
      else
         assign the common input tag to the gate output;
      endif
      remove tags from gate inputs;
      if(the gate output is a primary output)
         push the gate in output_stack;
      else
         line_status of gate output = STEM;
         put the node driven by the gate output in
         nodes_to_process list;
      endif
   end if
endwhile
fetch last node from nodes_to_process list;
if(node is a stem)
   fob_counter of stem = number of fan. br. of the stem;
   for(each fanout branch of the stem)
      push the stem in output_stack;
   endfor
   remove stem tag;
elseif(node is a gate)
   if(at least two inputs have different tags)
      node_status of gate = PRIM_RECONV_GATE;
      set prg_flag;
   endif
   remove tags from gate inputs;
   push the gate in output_stack;
endif
if(prg_flag not set)
   STEM is non-reconvergent;
   return();
endif
endstep


Step2(STEM)
beginstep
copy output_stack to common_stack;
while(common_stack not empty)
   pop node from common_stack;
   if(node is a stem)
      put stem in reencountered_stems list;
      decrement fob_counter of stem;
      if(fob_counter of stem == 0)
         node_status of stem = REMOVED;
         push node driving the stem in common_stack;
      endif
   elseif(node is a gate)
      if(node_status of gate == PRIM_RECONV_GATE)
         put gate in closing_reconvergence_gates list;
      else
         node_status of gate = REMOVED;
         for(each gate input)
            if(line_status of input == STEM)
               push node driving the gate input
               in common_stack;
            endif
         endfor
      endif
   endif
endwhile
for(each stem in the reencountered_stems list)
   if(fob_counter of stem != 0 and stem is reconvergent)
      for(each gate in closing_reconvergence_gates
      list of stem)
         if(node_status of gate!= PRIM_RECONV_GATE)
            node_status of gate = SEC_RECONV_GATE;
            set sec_reconv_flag;
         endif
      endfor
   endif
endfor
endstep


Step3(STEM)
beginstep
if(sec_reconv_flag set)
   for(each stem in reencountered_stems_list)
      fob_counter of stem = number of fan. br. of stem;
   endfor
   empty closing_reconvergence_gates list;
   copy output_stack to common_stack;
   while(common_stack not empty)
   pop node from common_stack;
      if(node is a stem)
         decrement fob_counter of stem;
```

```
        if(fob_counter of stem == 0)
           node_status of stem = REREMOVED;
           push node driving the stem in common_stack;
        endif
     elseif(node is a gate)
        if(node_status of gate == PRIM_RECONV_GATE)
           put gate in closing_reconvergence_gates list;
        else
           node_status of gate = REREMOVED;
           for(each gate input)
              if(line_status of input == STEM)
                 push node driving the gate input
                 in common_stack;
              endif
           endfor
        endif
     endif
  endwhile
endif
store closing_reconvergence_gates list in
closing_reconvergence_gates list of STEM;
endstep

Step4(STEM)
beginstep
for(each stem in reencountered_stems list)
   if(fob_counter of stem > 0 and fob_counter
      of stem < number of fanout branches of stem)
      for(each fanout branch of the stem)
         if(the fanout branch drives a primary output)
            add branch to exit_lines list;
         elseif(sec_reconv_flag not set and node_status
            of node driven by the fob == REMOVED)
            add branch to exit_lines list;
         elseif(sec_reconv_flag set and node_status
            of node driven by the fob == REREMOVED)
            add branch to exit_lines list;
         endif
      endfor
   endif
endfor
for(each gate in closing_reconvergence_gates list)
   add gate output to exit_lines list;
endfor
store exit_lines list in exit_lines list of STEM;
endstep
```

References

[1] M. Abramovici, P. R. Menon, and D. T. Miller, "Critical path tracing—An alternative to fault simulation," in *Proc. 20th Design Automation Conf.*, pp. 214–220, 1983.

[2] K. J. Antreich and M. H. Schulz, "Fast fault simulation in combinational circuits," in *Proc. Int. Conf. on Computer-Aided Design*, pp. 330–333, 1986.

[3] D. B. Armstrong, "A deductive method for simulating faults in logic circuits," *IEEE Trans. Comput.*, vol. C-21, pp. 464–471, 1972.

[4] F. Brglez, "A fast fault grader: Analysis and applications," in *Proc. Int. Test Conf.*, pp. 785–794, 1985.

[5] F. Brglez and H. Fujiwara, "A neutral netlist of 10 combinational benchmark circuits and a target translator in Fortran," in *Proc. Int. Symp. on Circuits and Systems*, pp. 663–698, 1985.

[6] D. Harel, R. Sheng, and J. Udell, "Efficient single fault propagation in combinational circuits," in *Proc. Int. Conf. on Computer-Aided Design*, 1987.

[7] D. Harel and B. Krishnamurthy, "Is there hope for linear time fault simulation?," in *Proc. 17th Fault Tolerant Computing Symp.*, pp. 28–33, 1987.

[8] D. Harel, M. Friedman, F. Maamari, and J. Rajski, "A dominators view of stem regions in combinational logic and its application to fault simulation," Tech. Report 87-50, Computer Res. Lab., Tektronix Lab., 1987.

[9] S. J. Hong, "Fault simulation strategy for combinational logic networks," in *Proc. 8th Int. Symp. on Fault Tolerant Computing*, pp. 96–99, 1978.

[10] F. Maamari and J. Rajski, "Reconvergent fanout analysis and fault simulation complexity of combinational circuits," *Inst. Elect. Eng. Electron. Lett.*, vol. 23, no. 21, pp. 1131–1133, 8th Oct. 1987.

[11] ——, "A reconvergent fanout analysis for efficient exact fault simulation of combinational circuits," in *Proc. 18th Fault Tolerant Computing Symp.*, pp. 122–127, June 1988.

[12] ——, "A fault simulation method based on stem regions," in *Proc. ICCAD-88*, pp. 170–173, Nov. 1988.

[13] E. G. Ulrich and T. Baker, "The concurrent simulation of nearly identical digital networks," in *Proc. 10th Design Automation Workshop*, vol. 6, pp. 145–150, 1973.

[14] J. A. Waicukauski, E. B. Eichelberger, D. O. Forlenza, E. Lindbloom, and T. McCarthy, "A statistical calculation of fault detection probabilities by fast fault simulation," in *Proc. Int. Test Conf.*, pp. 779–784, 1985.

[15] D. T. Wang, "An algorithm for the generation of test sets for combinational logic networks," *IEEE Trans. Comp.*, vol. C-24, pp. 742–746, July 1975.

[16] S. Seshu, "On an improved diagnosis program," *IEEE Trans. Electron. Comput.*, vol. EC-12, pp. 76–79, 1965.

*

Fadi Maamari received the B.Eng. and M.A.Sc. from École Polytechnique de Montréal, Montreal, P.Q., Canada, in 1984 and 1986, respectively. He is currently working towards the Ph.D. degree in electrical engineering in the VLSI Design Laboratory at McGill University, Montreal.

His research interests include CAD tools for VLSI testing, simulation, and design for testability.

*

Janusz Rajski (M'87) received the M.Eng. degree in electrical engineering from the Technical University of Gdańsk, Poland, in 1973, and the Ph.D. degree in electrical engineering from the Technical University of Poznań, Poland, in 1982.

He is currently an Associate Professor in the Department of Electrical Engineering at McGill University, Montreal, P.Q., Canada. His research includes testing and diagnosis, design for testability, logic synthesis, simulation, and computer architecture.